PHYD57H3 LEC01, Fall 2025
Machine Learning from very noisy data

Instructor: Prof. Pawel Artymowicz

Group 2 – [Yinglong (Jack) Zheng, 1007931277, yinglong.zheng@mail.utoronto.ca
Yiyang Xu, 1008887732 , yiyangflarsky.xu@mail.utoronto.ca
Hongyuan Ji,1006099026 , hongyuan.ji@mail.utoronto.ca
Weihao Wu ,1006959944, weihao.wu@mail.utoronto.ca]

1.0 Introduction

Neural networks are powerful nonlinear function approximators capable of learning complex relationships from data. According to the Universal Approximation Theorem (UAT), a feed-forward neural network with at least one hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy. This theoretical guarantee provides the foundational justification for using neural networks in financial time-series modeling, where the mapping between historical features and future price movements is highly nonlinear. In Section 1.1, we further elaborate on the mathematical formulation of the UAT and discuss why it supports the use of multilayer perceptrons in this project.

1.1. Universal Approximation Theorem

The universal approximation theorem (UAT) is one of the key mathematical results that justify using neural networks for function approximation. In its classical form, Cybenko proved that a feed-forward network with a single hidden layer, a sigmoidal activation function, and finitely many hidden units can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy, provided the weights and biases are chosen appropriately (Cybenko,1989). Shortly afterwards, Hornik and co-authors extended the result and showed that the universal approximation property is not tied to a specific activation such as the sigmoid, but rather to the multilayer feed-forward architecture itself: standard networks with any bounded, non-constant activation function are universal approximators (Hornik,1991).

Formally, let $C(K)$ denote the space of continuous real-valued functions on a compact set $K \subset \mathbb{R}^n$. The theorem states that for any $f \in C(K)$ and any ε>0, there exists a neural network of the form

$$F(x) = \sum_{i=1}^{N} v_i \sigma(w_i x + b_i) \qquad (1)$$

such that $|F(x) - f(x)| < \varepsilon$ for all $x \in K$ (Cybenko,1989). Here σ is a non-polynomial activation function (e.g. sigmoid, tanh, ReLU), and the parameters $\{v_i, w_i, b_i\}$represent the weights and biases of a single hidden-layer network. In practice this means that, at least in principle, a sufficiently wide multilayer perceptron (MLP) can represent very complicated nonlinear relations between inputs and outputs.

In this project, the unknown function is expected to approximate mapping a short history of market microstructure variables to the next-minute movement of the ETF price. After feature engineering and normalization, each training sample can be viewed as an input vector $x_t \subset \mathbb{R}^{NT}$, containing the past T one-minute observations of N different features (such as mid-price, turnover, forward return, and so on). The corresponding target vector is $y \in \mathbb{R}^M$ describing the next-minute data, where M denotes the dimension of the output. On short horizons, this mapping can be treated as a (piecewise) continuous function of the recent state of the limit-order book and trading activity. The UAT tells us that there exists some set of weights for our MLP that can approximate the conditional expectation $f(x_t) \approx E[y_t|x_t]$ arbitrarily well, assuming the data-generating process is sufficiently stationary over the training period.

A deep feed-forward network, which is also called a multilayer perceptron (MLP)—is the prototypical architecture in deep learning. In a DFF network, information flows in one direction, from input to output through a sequence of hidden layers, without any recurrent connections. Each layer performs an affine transformation followed by an element-wise nonlinearity:

$$h^{(1)} = \phi(W^{(1)}x + b^{(1)}) \quad (2)$$

$$h^{(l)} = \phi(W^{(l)}h^{(l-1)} + b^{(l)}) \quad (3)$$

$$\hat{y} = W^{(L)}h^{(L-1)} + b^{(L)} \quad (4)$$

where x is the input vector, $\phi(\cdot)$ is a nonlinear activation function (e.g. ReLU), and $\hat{y}$ is the network output (Goodfellow,2016). By stacking multiple such layers, the model constructs increasingly abstract representations of the input, which is why an MLP can capture nonlinear interactions that are impossible for a single linear model. The expressivity of DFF networks is closely tied to the UAT discussed above: a single sufficiently wide hidden layer is already universal, and deeper architectures can often reach similar approximation accuracy with fewer units per layer and better inductive biases (Goodfellow,2016). In practice, deep feed-forward networks are widely used as baselines for financial time-series forecasting and stock-return prediction. It often competes with more specialized sequence models such as LSTMs or Transformers.

1.2 Activative Function

1.2.1 ReLU Function

The Rectified Linear Unit (ReLU) is a widely used non-linear activation function defined as:

$$f(x) = max(0, x) \quad (5)$$

ReLU is piecewise linear and outputs the input when it is positive, and zero otherwise. Its simplicity provides several training advantages:

- It mitigates the vanishing gradient problem, since its gradient is 1 for all positive inputs, enabling faster and more stable backpropagation.
- It is computationally efficient, avoiding the costly exponential operations required by Sigmoid or Tanh.
- It promotes sparsity, because negative inputs produce zero activations, leading to more efficient and biologically inspired representations.

1.2.2 Sigmoid Function

$$sigmoid(x) = \frac{1}{1+e^{-x}} \quad (6)$$

The sigmoid function maps any real-valued input to the interval[0,1], giving it a natural probabilistic interpretation. It is smooth and differentiable everywhere, which facilitated early gradient-based learning methods. However, its derivative approaches zero for large positive or negative inputs, causing the vanishing-gradient problem and slowing or even preventing effective training in deep networks. Moreover, by compressing all inputs into a narrow output range, the sigmoid diminishes relative differences between values, limiting its ability to preserve informative signals in deeper layers.

1.2.3 Tanh

The Tanh function maps inputs to [−1,1] and is zero-centered, which generally improves optimization compared to the sigmoid. Its balanced output allows gradients to flow in both directions, often leading to faster convergence. However, like the sigmoid, Tanh saturates for large input magnitudes, causing vanishing gradients and limiting its effectiveness in deeper networks.

1.3 Initialization

1.3.1 He Initialization

He initialization is a widely adopted weight initialization strategy for neural networks that use ReLU activations.

$$W \sim N(0, \frac{2}{input}) \quad \textbf{(7) (He, 2015)}$$

ReLU discards negative inputs, it reduces the variance of signals passed between layers, which can lead to vanishing or unstable gradients when standard initialization is used. He initialization

compensates for this by scaling the initial weights according to the number of input connections, ensuring that both activations and gradients maintain stable variance throughout the network (He et al., 2015). This significantly improves training stability, making He initialization the default choice for deep ReLU-based architectures.

2.0 Method

2.1 First Model With Gradient Decent

2.1.1 Forward Propagation

Forward propagation involves passing a sequence of past feature vectors through successive network layers, each performing a linear transformation followed by a nonlinear activation. For a given layer, the pre-activation is computed as $Z = XW + b$, and the ReLU activation $ReLU(Z) = max(0, Z)$ (Shadbolt & Taylor, 2002), is applied elementwise. The combination of linear transformations and ReLU nonlinearities enables the network to represent complex, piecewise-linear functions, making it well suited for approximating the nonlinear dynamics observed in financial time series data. For the complete implementation details, please refer to Appendix A.

2.1.2 Loss Function

$$L = \frac{1}{N}\sum_{i=1}^{N}(\widehat{y}_i - y_i)^2,\ MSE = \frac{1}{2}(\widehat{y}_i - y_i)^2 \quad (8)$$

In regression tasks, the Mean Squared Error (MSE) is a standard loss function because it provides a continuous and differentiable measure of the discrepancy between predicted and true values. Its quadratic penalty amplifies larger errors, encouraging the model to correct substantial deviations more aggressively. Furthermore, the gradient of MSE is linear in the prediction error, yielding stable and computationally efficient backpropagation.

2.1.3 Backward Propagation

Backward propagation adjusts the model parameters by propagating the loss gradient from the output layer back through the network. Starting from the Mean Squared Error (MSE), the gradient with respect to the output is computed and passed backward through each layer using the chain rule. At each layer, the gradient is multiplied by the transpose of the weight matrix and modulated by the derivative of the ReLU activation to determine each neuron's contribution to the overall error. This produces parameter gradients of the form:

$$dW = A^T dZ,\ db = \Sigma dZ \quad (9)$$

which specify how the weights and biases should be updated. Gradient descent then adjusts the parameters according to

$$W_{new} = W_{old} - \eta dW, \quad (10)(\text{Shadbolt \& Taylor, 2002})$$

iteratively guiding the network toward a mapping that more accurately captures market dynamics. For the complete implementation details, please refer to Appendix A.

2.1.4 Data Choosing/Analisys

My model aims to predict future price movements to support investment decisions. Accordingly, my feature selection and preprocessing are based on principles from financial econometrics and financial machine learning.

First, price is the primary state variable that reflects all available market information in real time, making it essential for any forecasting task. Second, I include volume as a feature. It reflects the intensity of market participation, although it does not capture dollar value directly. Additionally, I use one-step return to represent the first-order difference of price, enabling the model to learn local momentum and short-term directional changes.

For preprocessing, I apply a log-transformation to volume, which greatly reduces the scale differences among the features. Without this transformation, volume is several orders of magnitude larger than price or returns, and this discrepancy leads to gradient explosion, numerical overflow under float32

precision, and instability in the hidden layers. These issues make training effectively impossible. In practice, the model becomes both trainable and stable only after the log-transformation is applied.
My model performs a time-series forecasting task because it predicts future market movements using historical price, volume, and return information. However, the model architecture itself is a feed-forward multilayer perceptron (DFF), which does not have inherent sequential memory. To encode temporal dependencies, I apply a rolling window approach that flattens the past 500 time steps into a fixed-length input vector. Thus, the model can be described as a "sliding-window DFF for time-series forecasting.

2.2 Second model with Nelder–Mead

Unlike the other models, this implementation does not use back-propagation. Instead, the neural network is trained by a derivative-free optimization method, the Nelder–Mead simplex algorithm, which searches for a local minimum of the loss function by iteratively deforming a simplex in parameter space.

The input data consist of intraday stock records with high, low and volume at each time step. From these raw values three features are constructed: Mid-price p_t,Turnover v_t and Forward return r_t.

$$p_t = \frac{1}{2}(High_t + Low_t) \quad (11)$$

$$v_t = p_t \times Volume_t \quad (12)$$

$$r_t = \frac{p_{t+1} - p_t}{p_t} \quad (13)$$

Each feature is then normalized by a hybrid log–z-score scheme.The corresponding feature-construction and normalization code is shown in Appendix B, Figure B.1. After shifting to avoid non-positive values, we apply log(1+x) and then standardize to zero mean and unit variance. This suppresses extreme outliers and keeps the different features on a comparable scale, which is important for a geometry-based method like Nelder–Mead.
The prediction model is a fully connected feed-forward network with two hidden layers and ReLU activations. A compact implementation of this forward pass is provided in Appendix B, Figure B.2. For each input window of 50 consecutive minutes, the network outputs three normalized quantities at the next time step: the mid-price, turnover and forward return. The training loss is the mean-squared error between predicted and true targets, averaged over all windows.
Let $\theta \in \mathbb{R}^n$ denote the vector of all trainable parameters in one block of the network and L(θ) the MSE loss. Nelder–Mead keeps a simplex of n+1 candidate parameter vectors $\{\theta^{(0)}, \ldots\ldots, \theta^{(n)}\}$. At each iteration the loss is evaluated at all vertices, they are sorted from best to worst, and the algorithm computes the centroid of all vertices except the worst one. Around this centroid it applies a sequence of geometric updates:Reflection of the worst vertex across the centroid; Expansion further in the same direction if reflection over-improves; Contraction toward the centroid if reflection fails; Shrink of the whole simplex toward the best vertex if none of the above steps reduces the loss (see Appendix B, Figure B.3) . These operations only require function evaluations and no gradients, making Nelder–Mead attractive as a gradient-free training method (Kincaid & Cheney, 2009).
However, classic Nelder–Mead degrades rapidly in very high dimensions: the simplex can no longer approximate the local curvature of the loss surface reliably. To address this, the network parameters are split into three blocks corresponding to the layers (W_1, b_1), (W_2, b_2), (W_3, b_3). The algorithm optimizes one block at a time while keeping the remaining blocks fixed. This block-wise training reduces the effective dimension of each optimization problem and yields much more stable behavior.

2.3 Third Model With Gradient-Free Random Search Optimization

Unlike the gradient-based and Nelder–Mead models used elsewhere in our group, our neural network is trained using a zero-order optimization method, Stochastic Random Search, which does not rely on

gradients. Instead of computing backpropagation or constructing a simplex, the algorithm repeatedly perturbs the model parameters with small random updates and accepts a change only if it reduces the loss. This procedure functions as a simple stochastic hill-climbing strategy.

2.3.1. Feature Construction

From the intraday QQQ dataset, we create five input features:

- Close price
- Volume
- Typical Price: $TP_t = \frac{H_t + L_t + C_t}{3}$
- VWAP up to time t: $VWAP_t = \frac{\sum_{i \le t} TP_i V_i}{\sum_{i \le t} V_i}$
- Price minus VWAP (a simple trend indicator)

To include time-series structure, we take a 10-minute window of these features and flatten it into a 50-dimensional vector. The target is the next-minute close price.
All features are standardized using z-score normalization.

2.3.2 Model Architecture

The prediction model is a small MLP:

- Input layer: 50 units
- Hidden layer: 20 units, ReLU activation
- Output layer: 1 value (the predicted next-minute close price)

The forward pass is:

$$z_1 = XW_1 + b_1, a_1 = ReLU(z_1),\ \hat{y} = a_1 W_2 + b_2$$

The loss function is mean squared error (MSE). This gives us a simple regression setup.

2.3.3. Random Search Training Method

Because gradients are unavailable in this setting, we train the network using a simple Random Search procedure. At each iteration:

- We take the current parameters: $\theta = \{W_1, b_1, W_2, b_2\}$
- Add small random noise: $\theta' = \theta + \varepsilon$ where ε comes from a normal distribution.
- Compute the loss using θ'
- If the loss becomes smaller, we keep θ'. Otherwise, we discard it.

This is repeated for 18,000 iterations. (Bergstra & Bengio, 2012)

2.4 Fourth Model With Adam Optimizer

The fourth model adopts the same forward and backward propagation structure as the first model, but it differs in both the choice of input features and the gradient optimization method. It uses a 30-minute sliding window, where 29 minutes of historical data are used as input and the model predicts the next 1 minute price. The model has two hidden layers with 192 nodes(can be changed).
The input features were chosen to form multiple dimensions of market behavior. Firstly, the model includes the Volume Modified Score(VMS), which is a core price volume indicator. This is shown to reflect local deviations in trading intensity. To enhance the model's ability and to detect intraday dynamics, this model adds some additional features. For instance, rolling normalized price, log returns and MACD components. These inputs together encode trend, momentum, volatility, and mean reversion characteristics, allowing the neural network to learn a richer representation of the underlying market microstructure than using VMS alone.

2.4.1 Input Features

- Rolling VMS: For a rolling window of length w=30 minutes, we compute rolling mean and standard deviation of price and volume:

$$\overline{P_t} = mean(P_{t-w-1},..., P_t),\ \sigma_P(t) = std(P_{t-w-1},..., P_t) \quad (14)$$

$$\overline{V_t} = mean(V_{t-w-1},..., V_t),\ \sigma_P(t) = std(P_{t-w-1},..., P_t) \quad (15)$$

To avoid division by zero, any zero standard deviation is replaced by 1. The VMS feature is then

$$VMS_t = \frac{(P_t - \overline{P})}{\sigma_P} \cdot \frac{(V_t - \overline{V})}{\sigma_V} \quad (16)$$

- Rolling Normalized Price: Using the same rolling statistics, we define

$$P_{norm}(t) = \frac{P_t - \overline{P_t}}{\sigma_P(t)}, \quad (17)$$

which measures how far the current price is from its recent mean in units of local volatility.

- Log Return: The model uses log returns because it produces a stationary, additive, and mathematically well behaved representation of price changes. Which leads to more stable and reliable model training compared to simple percentage returns.

$$r_t = ln(\frac{P_t}{P_{t-1}}),\ t \geq 1 \quad (18)$$

with $r_0 = 0$. This encodes short-term price changes in a scale-invariant way.

- MACD(Moving Average Convergence Divergence): The model also includes MACD as an additional input feature because it captures price momentum and trend-reversal structure in a smoothed manner. This would provide information that is not directly visible in raw prices nor returns. Which helps the neural network learn more stable and predictive patterns.

$$\alpha_n = \frac{2}{n+1}$$

$$EMA_{12}(t) = \alpha_{12} P_t + (1 - \alpha_{12}) EMA_{12}(t-1) \quad (19)$$

$$EMA_{26}(t) = \alpha_{26} P_t + (1 - \alpha_{26}) EMA_{26}(t-1) \quad (20)$$

$$Signal(t) = \alpha_9 MACD_t + (1 - \alpha_9) Signal(t-1) \quad (21)$$

2.4.2 Adam Optimizer

This model trains the neural network using the Adam optimizer, which is a stochastic gradient–based method that adaptively adjusts the learning rate for each parameter(Goodfellow, Bengio, & Courville, 2016). Adam optimizer combines the advantages of momentum (first-order moment estimation) and RMSProp (second-order moment estimation). For parameter θ, with gradient g_t at step t, Adam updates are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (22)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g^2_t \quad (23)$$

with bias correction:

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t},\ \widehat{v}_t = \frac{v_t}{1-\beta_2^t} \quad (24)$$

The parameter update rule is:

$$\theta_t = \theta_{t-1} - \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon} \quad (25)$$

3.0 Results

3.1 Gradient Decent method for different iteration

During the training phase, I set the window size to 500, meaning that for each training sample the model reads the past 500 time steps of four feature categories: price, return, absolute return, and log-volume and concatenates them into a single input vector of length 4x500 Therefore, when max_N = 3000, the final window corresponds approximately to the interval [p2500,p3000] and is used to predict the next-step return. The parameter MAX_N determines the amount of data used for training, iteration specifies how many times gradient descent is performed, and the learning rate controls the magnitude of each parameter update. After training, I conduct recursive forecasting using the next 20 steps of realized volume under the assumption that volume is partially predictable.
The experimental results show that when iteration = 1, the model is clearly underfitting: it fails to adequately match the historical 3000 step return sequence(Figure 3.1) and produces poor forecasts for the next 20 step price trajectory(Figure 3.2).

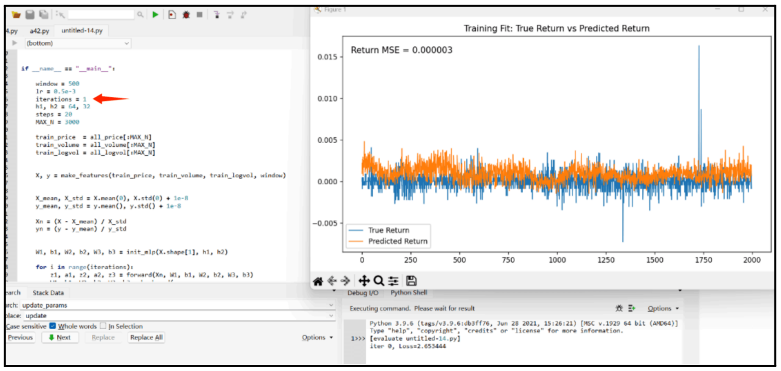


Figure 3.1(iteration = 1)



Figure 3.2(iteration = 1)

When iteration increases to 1000, the model begins to capture certain short-term structures, resulting in a markedly improved fit to historical returns(Figure 3.3), and the predicted 20 step price path becomes closer to the true trajectory(Figure 3.4).

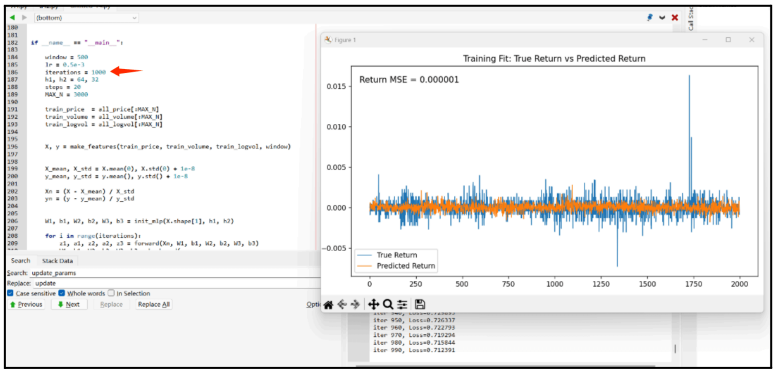


Figure 3.3(iteration = 1000)

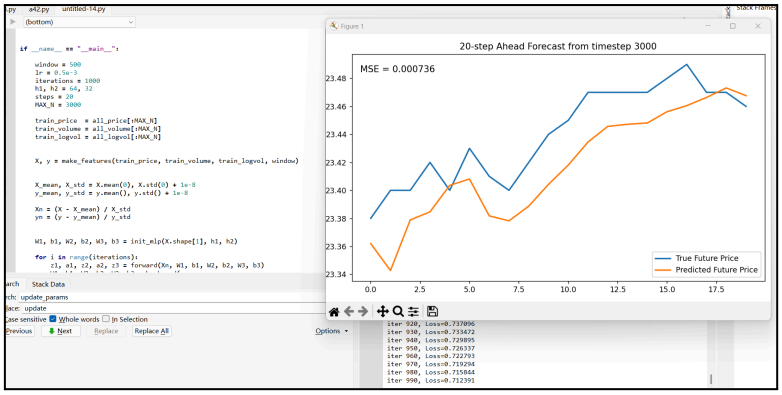


Figure 3.4(iteration = 1000)

However, when iteration is further increased to 100000, the model's fit on the training data continues to improve(Figure 3.5), but the mean squared error of the 20-step price forecast deteriorates(Figure 3.6).

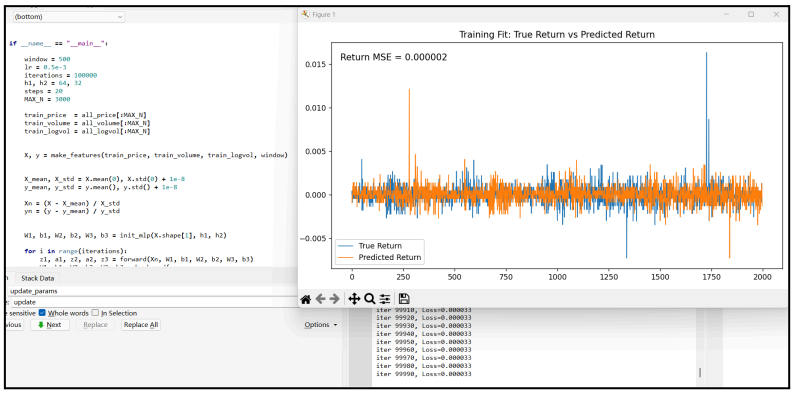


Figure 3.5(iteration = 100000)

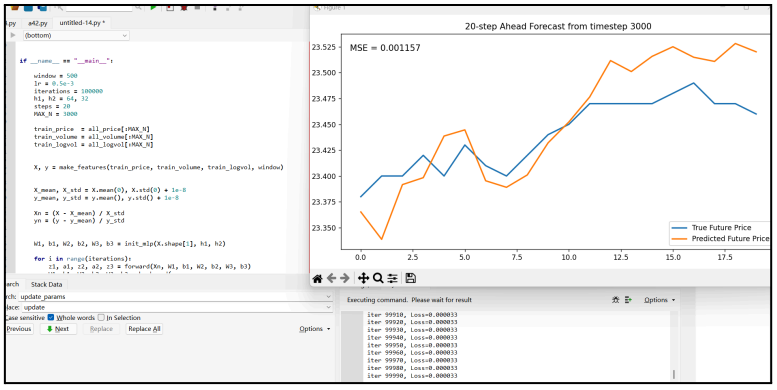


Figure 3.6(iteration = 100000)

This reflects a typical case of overfitting, where the model learns not only the underlying structure but also the noise in the training data, which harms generalization performance. Overall, a moderate number of iterations (such as 1000) achieves a better balance between learning useful structure and avoiding overfitting.

3.2 Second Model For Nelder–Mead with different numbers of ReLU units

For the Nelder–Mead–trained MLP, the program was run with 500-step and 50-step rollouts, under two different network sizes. Figures B.1 and B.2 (see Appendix B) correspond to the smaller model (32 units in the first hidden layer and 16 units in the second), while Figures B.3 and B.4 (see Appendix B) show the larger model (64 and 32 hidden units respectively).

In all cases, the predicted price path is much smoother than the true price. Over 50 steps the network roughly tracks the local level of the series, but it systematically underestimates short-term volatility and often lags behind turning points. Over 500 steps the effect becomes more pronounced: the forecasts exhibit a slow monotonic drift, whereas the actual price fluctuates around a relatively flat range. This behaviour is consistent with the scatter plots and sign-accuracy statistics (not shown here): the model has essentially learned to output a small, almost constant one-minute return, so that tiny biases accumulate into a visible trend over long horizons.

3.3 Third Model With Gradient-Free Random Search Optimization

After 18,000 iterations of Random Search, the model reached a reasonable training and testing loss. The last 200 minutes of predictions are shown in the figure. Although the predicted values do not match the real prices exactly, the model is able to follow the general trend, especially around local movements. The increasing gap between the real and predicted values is expected

3.4 Results for Fourth Model With Adam Optimizer

Table 3.1: Quantitative summary of RMSE values.

Horizon(Steps)	RMSE	Behavior
10	6.46	Stable, small bias
18	31.76	Instability increases
50	NaN	Divergence

The fourth model evaluated by using recursive multi step forecasting on the 2022 QQQ dataset. Unlike one step prediction, the model relied solely on its own previous outputs, making the task more realistic and more challenging. The model is using 30 minutes as one window each learning step is one minute.

Overall, the model performs reasonably well for very short horizons (below 10 steps), producing stable predictions with limited drift. However, as the recursive horizon increases, forecasting accuracy deteriorates rapidly. The RMSE increases from approximately 6.46 (10-step) to 31.76 (18-step), and long horizon predictions (50-step) become numerically unstable and diverge.

These results are consistent with the theoretical difficulty of long horizon prediction in very noisy financial data, where error accumulation and model bias cause predictions to drift away from the true price path. Detailed plots and analysis are provided in 6.1.4 Appendix D.

4. Discussion

4.1 Reflections on Gradient Decent

It is important to clarify in advance that even the most advanced models available today perform only slightly better than random guessing when it comes to directional prediction in financial markets. Several fundamental reasons explain this. First, short-term market prices are dominated by extremely high noise, and the proportion of truly predictable structural signals is very small, making it inherently difficult for any model to extract stable predictive information. Second, mathematically, short-horizon price movements behave very close to a random walk, meaning that the underlying process contains little persistent structure to learn (Samuelson, 1965). In addition, the usefulness of single-step predictions is extremely short-lived—when the model's own predictions are fed back for multi-step inference, the accumulated errors grow rapidly and further degrade forecasting accuracy.

Although the model shows some ability to fit historical returns and produce short-horizon forecasts, several limitations significantly reduce its predictive reliability. First, the feature set is relatively narrow: it contains only price, return, absolute return, and log-volume, all of which are univariate transformations of a single asset. Financial markets are inherently multi-factor systems, and excluding

richer information such as volatility measures, technical indicators, cross-asset interactions, or macroeconomic variables, results in insufficient explanatory power. Consequently, the model's ability to capture meaningful structure is fundamentally constrained.

Second, the forecasting horizon is extremely short. One-minute returns are well-known to be almost entirely noise-driven and exhibit extremely weak autocorrelation. When a model is trained on such noisy targets, most of the patterns it learns inevitably correspond to noise rather than underlying economic dynamics. This reduces the stability of the fitted function and limits its generalization capability, especially when predictions are used recursively for multi-step forecasting.

Finally, the forecasting procedure uses realized future volume as an input. Although volume does exhibit partial predictability in a statistical sense, using its actual future values unavoidably introduces look-ahead bias. This creates an overly optimistic scenario in which the model benefits from information that would not be available in real-time decision-making. A more realistic forecasting framework would require an auxiliary model to predict future volume, rather than relying on realized values.

4.2 Reflections on Nelder–Mead–Trained MLP

The Nelder–Mead–trained MLP shows that, although the block-wise simplex procedure can reduce training MSE, its predictive power for one-minute QQQ.txt returns is very limited. Predicted forward returns are strongly concentrated near zero, and the sign accuracy is essentially at the 50% "coin-flip" level. When these small biases are rolled forward over 50–500 steps, the reconstructed price paths are overly smooth and display a slow drift, while the true price remains much more volatile and mean-reverting.

This behaviour is partly due to the data and partly due to the method. One-minute ETF returns are extremely noisy, so minimizing MSE naturally pushes the model toward predicting a nearly constant return instead of chasing random fluctuations. At the same time, Nelder–Mead is a derivative-free optimizer and struggles in the high-dimensional parameter space of a neural network; the algorithm can only explore a small simplex at a time and easily gets stuck in flat regions where the output is almost constant. Compared with our group's back-propagation models, which fit the training price path much more closely, this experiment highlights how important efficient gradient-based optimization is for deep learning on financial time series. The results also suggest that gradient-free methods alone are unlikely to uncover weak predictability at the one-minute horizon, although they could be useful as a complementary fine-tuning tool in lower-dimensional settings.

4.3 Reflections on Gradient-Free Random Search Optimization

4.3.1 Model strengths

Although very simple, this method works because: First, we only need loss values, not gradients. And our model is small enough that random exploration can find better parameters. Besides, the method is stable even when the data is noisy. This makes Random Search a good fit for a "no-backprop" assignment. (Bergstra & Bengio, 2012)

4.3.2 Limitations

- Accuracy drops quickly for longer horizons.
- The model may stagnate in flat regions of the loss.
- The feature set only includes QQQ itself.

4.3.3 Possible Improvements

Our model only uses QQQ-based features. However, QQQ is strongly driven by the price movements of its largest components (e.g., AAPL, MSFT, NVDA). Incorporating one of these stocks as an external input feature could potentially improve the prediction accuracy. Although we did not have time to implement this idea, it would be a valuable improvement for future versions of the model.

4.4 Reflection on the fourth model with Adam Optimizer

4.4.1 Why recursive forecasting produces drift and instability

This divergence phenomenon in the model's recursive forecasts is a direct consequence of error accumulation inherent to iterated multi step prediction. Because each predicted value is used for the

model as part of the next input window, any initial error compounds across subsequent steps. This mechanism is well documented in forecasting literature. Furthermore, an error made at one forecast horizon will be propagated to the subsequent forecast horizons, thus increasing the bias of the recursive strategy as the horizon increases(Ben Taieb & Hyndman(2012), p.16). This framework explains why short horizon recursive predictions (e.g., 10 steps) remain relatively accurate, while longer horizon predictions(18–50 steps) drift substantially from the true price trajectory and may even exhibit explosive behavior.

4.4.2 Why gradually increasing the forecast horizon reduces MSE

When the forecast horizon is increased step by step (10, 13, 15, 18), each experiment still begins with true historical data, which keeps the early predictions stable and prevents large drift. In contrast, generating all 18 steps in a single run will force the model into a long recursive chain without any correction. This would make the model more likely to leave the range of values seen during training. This aligns with the idea that long horizon forecasts are generally much less reliable and more error prone than short horizon ones(Hyndman & Athanasopoulos, 2021). Figures shown in Appendix D.

4.4.3 Feature Sensitivity

In the fourth model, many of the input features(VMS, normalized prices, MACD) depend nonlinearly on previous prices. A small deviation in the predicted series will cause disproportionate distortions in the feature space. This accelerates error growth during recursive forecasting and contributes to instability.

4.4.4 Limitation for Feed Forward Network

The feed-forward architecture lacks explicit temporal memory. Although the sliding window can encode short-term history, the model does not maintain a hidden state which keeps the memory. Consequently, the network cannot correct accumulated drift or distinguish between structural price movements and noise when used recursively.

5.0 Conclusion

Across all four models implemented in this project, gradient descent trained MLP, Nelder–Mead simplex MLP, gradient-free random search MLP, and the Adam optimized MLP. We consistently observed the same qualitative outcome, in which short-horizon predictions remain relatively stable, while long-horizon recursive forecasts rapidly deteriorate due to error accumulation.

Despite using fundamentally different optimization strategies and feature sets, each model exhibited the same characteristic drift when predictions were recursively fed back as inputs. This aligns with theoretical expectations described in the literature on multi-step forecasting, where recursive strategies inevitably propagate early errors into later horizons, amplifying bias and reducing stability (Ben Taieb & Hyndman, 2012, p.16).

5.1 Future Analysis

Looking forward, several directions appear promising. More advanced models such as LSTMs, GRUs, and Transformers incorporate explicit temporal memory, allowing them to preserve information over longer horizons and potentially reduce error propagation. Unlike static feed-forward models, recurrent architectures update their hidden state at every step, enabling them to learn temporal dependencies rather than relying solely on fixed sliding windows.

Overall, while the four models differed substantially in methodology, their consistent behavior under recursive forecasting provides strong evidence that the challenge lies not primarily in the optimization method, but in the predictive structure of the data and the limitations of memory-less architectures. More expressive sequence models represent a natural next step for improving multi-step forecasting in noisy financial environments.

6.0 References

1. **Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13, 281–305.**
2. **Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals and Systems, 2(4), 303–314. https://doi.org/10.1007/BF02551274**
3. **Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.**
4. **He, K., Zhang, X., Ren, S., & Sun, J. (2015).Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification.**
5. **Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts.**
6. **Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359–366.**
7. **Kincaid, D., & Cheney, W. (2009). Numerical analysis: Mathematics of scientific computing (3rd ed.). American Mathematical Society.**
8. **Shadbolt, N., & Taylor, J. G. (Eds.). (2002). *Neural networks and the financial markets: Predicting, combining and portfolio optimisation*. Springer.**
9. **Samuelson, P. A. (1965). Proof that properly anticipated prices fluctuate randomly. Industrial Management Review, 6(2), 41–49.**
10. **Taieb, S. B., & Hyndman, R. J. (2014). Recursive and direct multi-step forecasting: The best of both worlds. International Journal of Forecasting, 30(2), 177–188.**

6.1 Appendix

6.1.1 Appendix A: First Model

```
def relu(x):
    return np.maximum(0, x)

def relu_grad(x):
    return (x > 0).astype(float)
```

Figure A.1: activation function ReLU and derivative of ReLU

```
def init_weight(input_dim, h1=64, h2=32):
    W1 = np.random.randn(input_dim, h1) * np.sqrt(2 / input_dim)
    b1 = np.zeros((1, h1))

    W2 = np.random.randn(h1, h2) * np.sqrt(2 / h1)
    b2 = np.zeros((1, h2))

    W3 = np.random.randn(h2, 1) * np.sqrt(2 / h2)
    b3 = np.zeros((1, 1))

    return (W1, b1, W2, b2, W3, b3)
```

Figure A.2: Initialize the weight matrices using He initialization

```
def forward(X, W1, b1, W2, b2, W3, b3):
    z1 = X @ W1 + b1
    a1 = relu(z1)

    z2 = a1 @ W2 + b2
    a2 = relu(z2)

    z3 = a2 @ W3 + b3   #Since this is a regression model, we don't need to relu the output
    return z1, a1, z2, a2, z3
```

Figure A.3: Forward propagation

```
def backward(X, y, z1, a1, z2, a2, z3,
             W1, b1, W2, b2, W3, b3, lr):

    m = y.shape[0]

    dz3 = 2 * (z3 - y.reshape(-1, 1)) / m
    dW3 = a2.T @ dz3
    db3 = dz3.sum(0, keepdims=True)

    da2 = dz3 @ W3.T
    dz2 = da2 * relu_grad(z2)
    dW2 = a1.T @ dz2
    db2 = dz2.sum(0, keepdims=True)

    da1 = dz2 @ W2.T
    dz1 = da1 * relu_grad(z1)
    dW1 = X.T @ dz1
    db1 = dz1.sum(0, keepdims=True)

    W1 -= lr * dW1
    b1 -= lr * db1
    W2 -= lr * dW2
    b2 -= lr * db2
    W3 -= lr * dW3
    b3 -= lr * db3

    return W1, b1, W2, b2, W3, b3
```

Figure A.4: Backward propagation

6.1.2 Appendix B: Second Model

```
def build_X_y(df, window_steps=50, step=1, num_windows=None):
    p = price(df).astype(np.float32)
    t = turnover(df).astype(np.float32)
    f = forward_pct(df).astype(np.float32)

    def hybrid_norm(x):
        x_shift = x - np.min(x) + 1e-8
        x_log = np.log1p(x_shift)
        return (x_log - x_log.mean()) / (x_log.std() + 1e-8)

    p_n, t_n, f_n = hybrid_norm(p), hybrid_norm(t), hybrid_norm(f)
    N = len(p_n)

    X, y = [], []

    max_start = N - window_steps - 1
    if num_windows is None:
        num_windows = max_start // step

    k = 0
    for _ in range(num_windows):
        if k + window_steps + 1 >= N:
            break

        X_win = np.concatenate([
            p_n[k:k+window_steps],
            t_n[k:k+window_steps],
            f_n[k:k+window_steps]
        ])
        y_win = np.array([
            p_n[k + window_steps],
            t_n[k + window_steps],
            f_n[k + window_steps],
        ])
        X.append(X_win)
        y.append(y_win)
        k += step

    return np.array(X, dtype=np.float32), np.array(y, dtype=np.float32)
```

Figure B.1 Python implementation of feature construction and the hybrid log–z-score normalization used to build the training windows.

```
def relu(x):
    return np.maximum(0, x)

def forward(X, params):
    W1, b1, W2, b2, W3, b3 = params
    A1 = relu(np.dot(X, W1) + b1)
    A2 = relu(np.dot(A1, W2) + b2)
    Y_pred = np.dot(A2, W3) + b3
    return Y_pred

def mse_loss(y_pred, y_true):
    return np.mean((y_pred - y_true)**2)
```

Figure B.2 Forward-pass implementation of the two-hidden-layer ReLU network used as the prediction model.

```
def nelder_mead_momentum(loss_fn, x0,
                                                alpha=1.0,      # reflection
                                                gamma=2.0,      # expansion
                                                beta=0.5,       # contraction
                                                shrink=0.5,     # shrink
                                                momentum=0.3,
                                                max_iter=200,
                                                tol=1e-5):

    n = len(x0)


    simplex = [x0]
    for i in range(n):
        x = x0.copy()
        x += np.random.uniform(-0.1, 0.1, size=n)
        simplex.append(x)
    simplex = np.array(simplex)

    fvals = np.array([loss_fn(x) for x in simplex])
    prev_update = np.zeros_like(x0)

    for it in range(max_iter):

        if it % 10 == 0:
            spread = np.linalg.norm(simplex[-1] - simplex[0])
            print(f"Iter {it}: simplex spread = {spread:.6f}")
```

```
idx = np.argsort(fvals)
simplex = simplex[idx]
fvals = fvals[idx]

best = simplex[0]
worst = simplex[-1]
second_worst = simplex[-2]


x_bar = np.mean(simplex[:-1], axis=0)

# ---------- Reflection ----------
xr = x_bar + alpha * (x_bar - worst)
fr = loss_fn(xr)

if fvals[0] <= fr < fvals[-2]:
        simplex[-1], fvals[-1] = xr, fr

elif fr < fvals[0]:
        # ---------- Expansion ----------
        xe = x_bar + gamma * (xr - x_bar)
        fe = loss_fn(xe)
        if fe < fr:
                simplex[-1], fvals[-1] = xe, fe
        else:
                simplex[-1], fvals[-1] = xr, fr
```

```
        else:
                # ---------- Contraction ----------
                xc = x_bar + beta * (worst - x_bar)
                fc = loss_fn(xc)
                if fc < fvals[-1]:
                        simplex[-1], fvals[-1] = xc, fc
                else:
                        # ---------- Shrink ----------

                        for i in range(1, len(simplex)):
                                simplex[i] = best + shrink * (simplex[i] - best)
                                fvals[i] = loss_fn(simplex[i])

        # ---------- Momentum step ----------
        if it > 0 and np.std(fvals) < tol * 10:

                update = best - x_bar
                momentum_step = momentum * (update + prev_update)
                simplex[-1] = simplex[-1] + momentum_step
                fvals[-1] = loss_fn(simplex[-1])
                prev_update = update
                print(f"Iter {it}: momentum step applied (best loss={fvals[0]:.6f})")


        if np.std(fvals) < tol:
                print(f"Converged at iter {it}, best loss = {fvals[0]:.6f}")
                break

        if it % 20 == 0:
                print(f"Iter {it}: best loss = {fvals[0]:.6f}")


return simplex[0], fvals[0]
```

Figure B.3 Block-wise Nelder–Mead training procedure. The full parameter vector is split into three blocks (W1,b1),(W2,b2),(W3,b3), and each block is optimized in turn using the simplex algorithm with momentum.

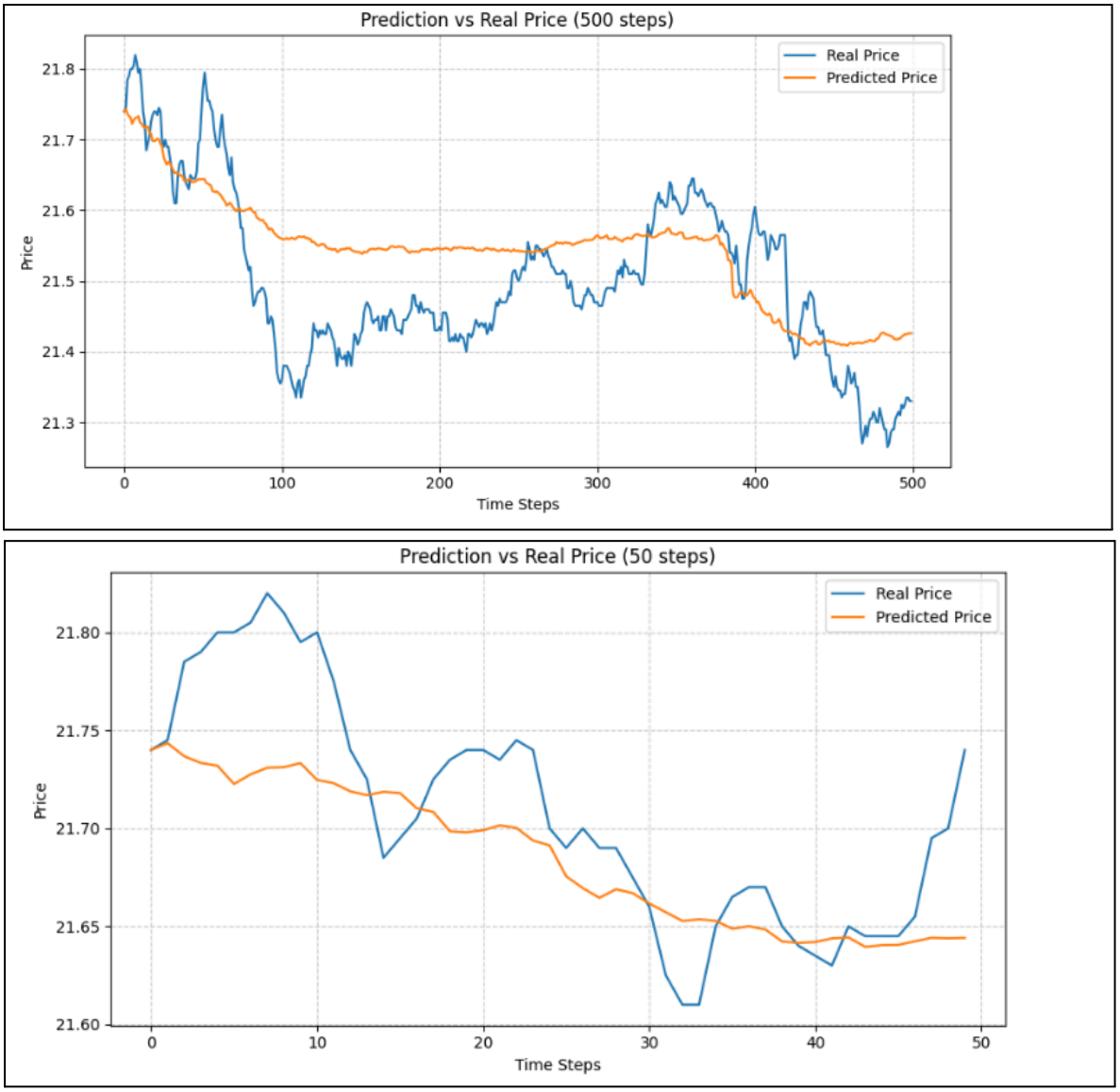
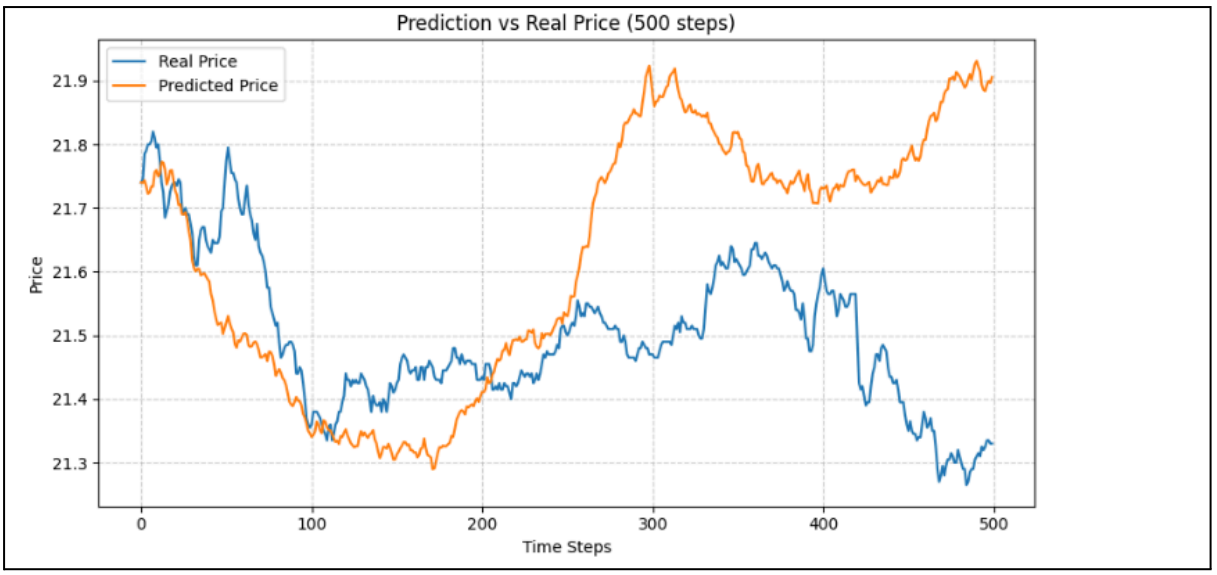


Figure B.4 and **Figure B.5**: Prediction vs Real Price Value Graph with 500 time steps and 50 time steps when the first hidden layers contain 32 artificial neurons and the second hidden layers contain 16 artificial neurons.



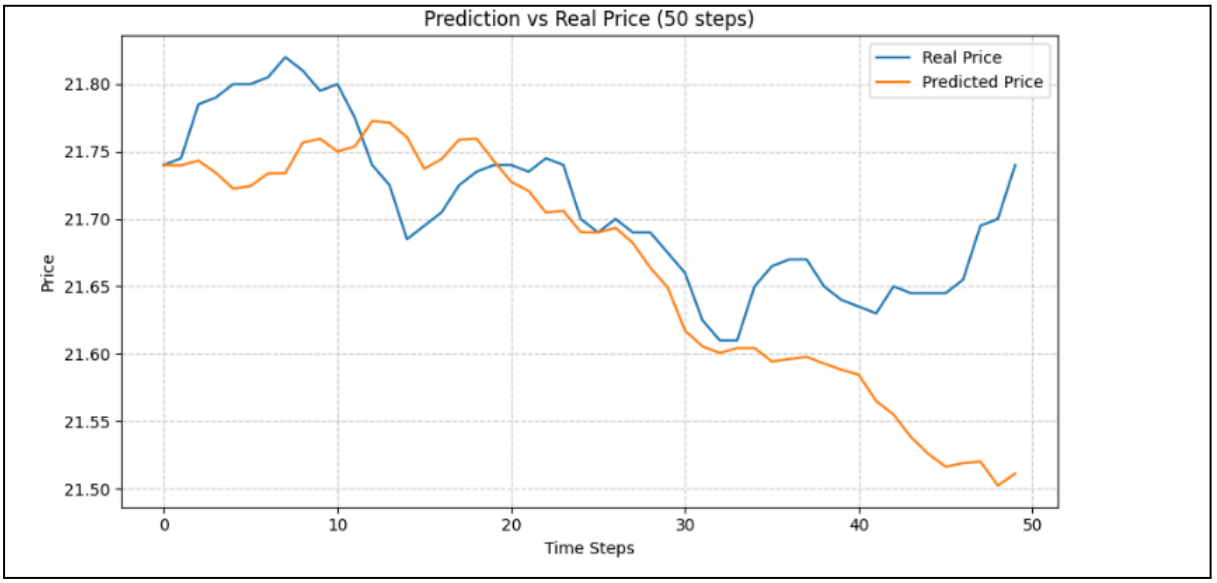


Figure B.6 and **Figure B.7**: Prediction vs Real Price Value Graph with 500 time steps and 50 time steps when the first hidden layers contain 64 artificial neurons and the second hidden layers contain 32 artificial neurons.

6.1.3 Appendix C: Third Model Results

6.1.3.1 The Third model result images shown below:

```
Iter 12500, Best Loss: 0.077918
Iter 13000, Best Loss: 0.076846
Iter 13500, Best Loss: 0.075905
Iter 14000, Best Loss: 0.075850
Iter 14500, Best Loss: 0.072197
Iter 15000, Best Loss: 0.071109
Iter 15500, Best Loss: 0.071109
Iter 16000, Best Loss: 0.070933
Iter 16500, Best Loss: 0.069983
Iter 17000, Best Loss: 0.069298
Iter 17500, Best Loss: 0.069035
Training Done!
Final Train Loss: 0.06680821643952789
Test Set MSE: 0.587077
Prediction example (last 5 mins):
True: [27.55 27.55 27.57 27.58 27.57]
Pred: [29.01436318 29.42757124 28.8880341  29.34140168 28.82782248]
```

Figure C.1: Training Log and Sample Predictions Final training loss, test MSE, and last-5-minute prediction example from the gradient-free MLP model.

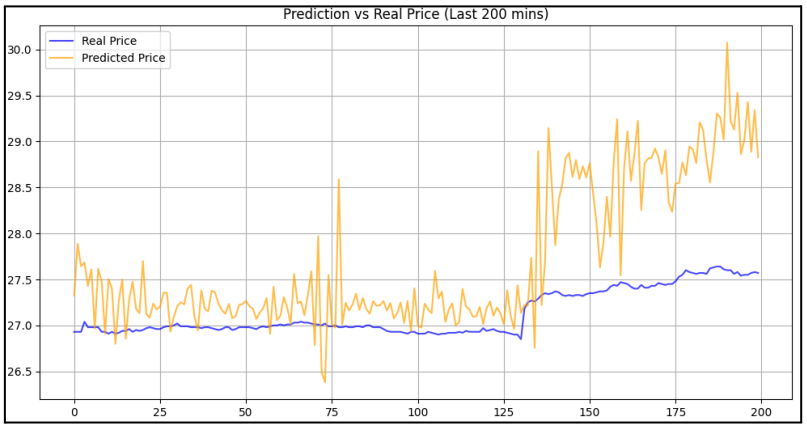


Figure C.2: Comparison of the model's predicted next-minute prices and the actual prices over the final 200 minutes of the test set, showing that the model captures general movement but not exact price levels.

6.1.3.2 Part code of random search

```python
# Training: Random Search
np.random.seed(555
               )

# Initialize 4 sets of parameters
# W1: Connect Input(50) to Hidden(20)
# Initialize weights with small random values to ensure the model's initial error is within a reasonable range.
# If initial weights are too large, prediction results will be unstable.
best_W1 = np.random.randn(input_dim, hidden_dim) * 0.1
best_b1 = np.zeros(hidden_dim)
# W2: Connect Hidden(20) to Output(1)
best_W2 = np.random.randn(hidden_dim) * 0.1
best_b2 = 0.0

# Package them into a list for easy passing to the forward function later
best_params = [best_W1, best_b1, best_W2, best_b2]

# Calculate initial loss once
y_init = forward_mlp(X_train, best_params)
best_loss = compute_loss(y_init, y_train)

print(f"Initial Loss: {best_loss:.6f}")

# Iterate 5000 times, random search step size 0.01
iterations = 18000
learning_step = 0.005

for i in range(iterations):
    # Generate Noise for the 4 parameters separately
    noise_W1 = np.random.randn(input_dim, hidden_dim) * learning_step
    noise_b1 = np.random.randn(hidden_dim) * learning_step
    noise_W2 = np.random.randn(hidden_dim) * learning_step
    noise_b2 = np.random.randn() * learning_step

    # Generate Candidates
    cand_W1 = best_W1 + noise_W1
    cand_b1 = best_b1 + noise_b1
    cand_W2 = best_W2 + noise_W2
    cand_b2 = best_b2 + noise_b2
    cand_params = [cand_W1, cand_b1, cand_W2, cand_b2]

    # Calculate candidate Loss (note that forward_mlp is used here)
    y_cand = forward_mlp(X_train, cand_params)
    candidate_loss = compute_loss(y_cand, y_train)

    # Compare: if better, then proceed
    if candidate_loss < best_loss:
        best_loss = candidate_loss
        best_params = cand_params
        # Update specific W and b for easier noise addition next time
        best_W1, best_b1, best_W2, best_b2 = cand_W1, cand_b1, cand_W2, cand_b2

    if i % 500 == 0:
        print(f"Iter {i}, Best Loss: {best_loss:.6f}")

print("Training Done!")
print("Final Train Loss:", best_loss)
```

6.1.4 Appendix D: Fourth Model

6.1.4.1 The fourth model result images shown below:

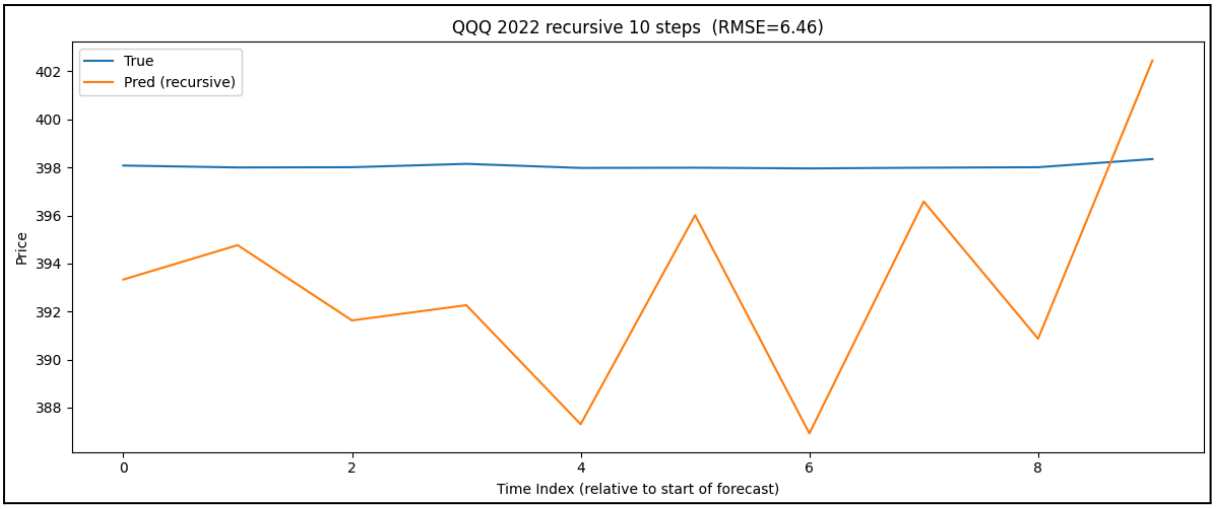


Figure D.1: Recursive 10-step forecast (RMSE = 6.46)

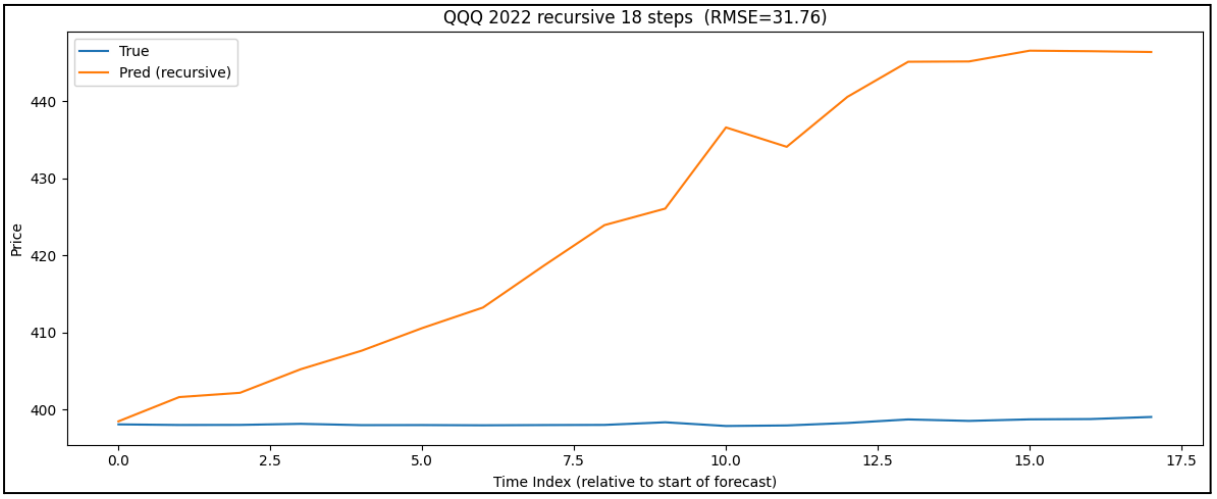


Figure D.2: Recursive 18-step forecast (RMSE = 31.76)

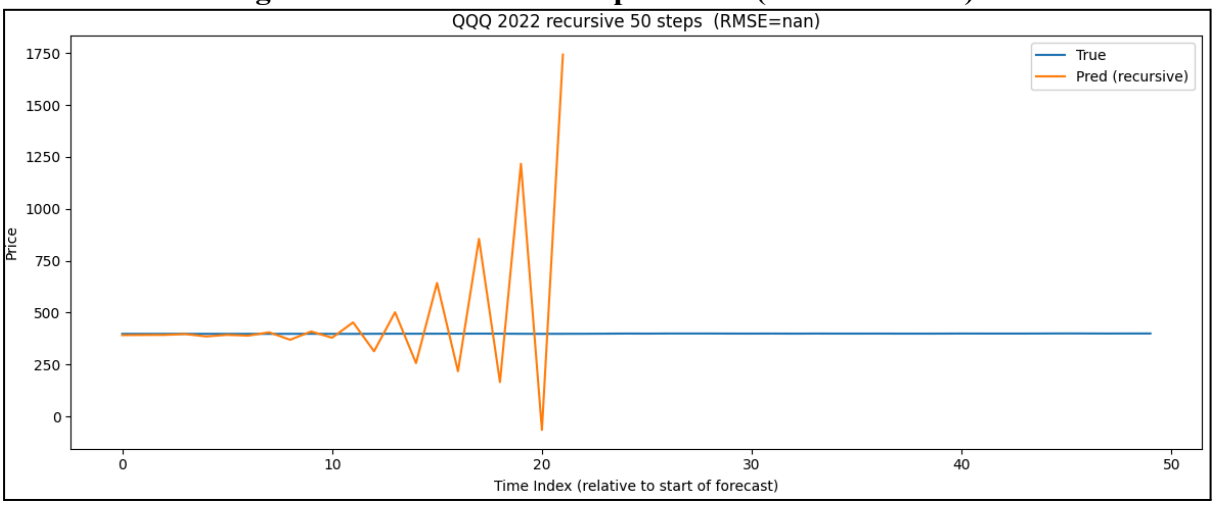


Figure D.3: Recursive 50-step forecast (Divergent, RMSE = NaN)

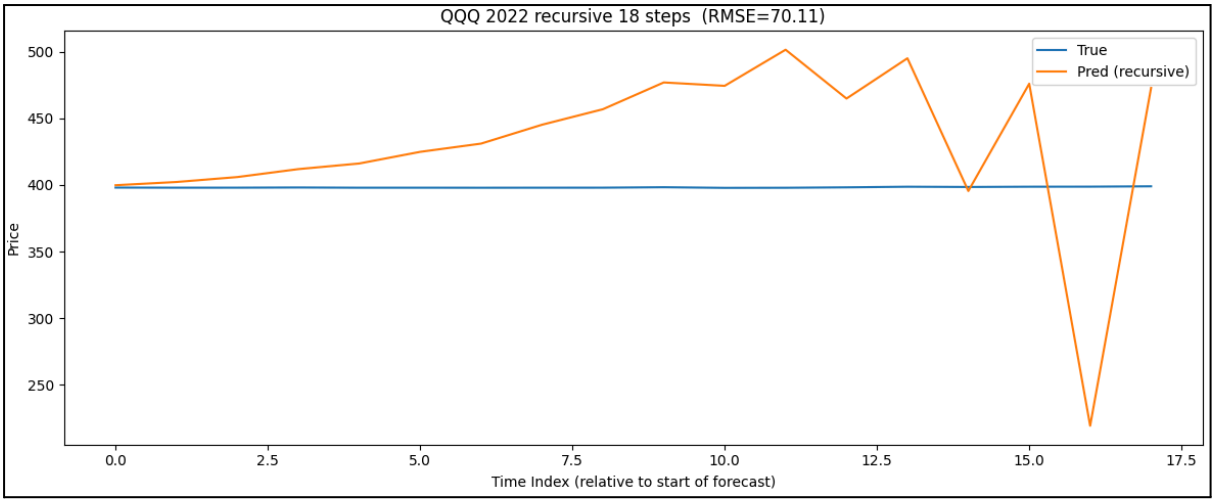


Figure D.4:Directly generate 18 steps forecast(RMSE=70.11)

Result for Figure D.4

Epoch 1/10, loss=0.011500
Epoch 2/10, loss=0.002265
Epoch 3/10, loss=0.001739
Epoch 4/10, loss=0.001135
Epoch 5/10, loss=0.001338
Epoch 6/10, loss=0.000945
Epoch 7/10, loss=0.000946
Epoch 8/10, loss=0.000968
Epoch 9/10, loss=0.000928
Epoch 10/10, loss=0.000663
Recursive RMSE = 70.10971402937629

6.1.4.2 Part of code for Adam Optimizer

```
def relu(x):
    return np.maximum(0, x)

def init_params(input_dim, h1=64, h2=32):
    return {
        "W1": 0.01 * np.random.randn(h1, input_dim),
        "b1": np.zeros(h1),
        "W2": 0.01 * np.random.randn(h2, h1),
        "b2": np.zeros(h2),
        "W3": 0.01 * np.random.randn(1, h2),
        "b3": 0.0,
    }

def forward(x, params):
    W1, b1 = params["W1"], params["b1"]
    W2, b2 = params["W2"], params["b2"]
    W3, b3 = params["W3"], params["b3"]
```

```
    # a1 = W1x + b1
    # h1 = ReLU(a1)
    # a2 = W2h1 + b2
    # h2 = ReLU(a2)
    # y_hat = W3h2 + b3
    a1 = W1 @ x + b1 # First layer affine
    h1 = relu(a1)
    a2 = W2 @ h1 + b2 # Second layer affine
    h2 = relu(a2)
    y_hat = W3 @ h2 + b3 # Output layer affine
    return y_hat.item(), (x, a1, h1, a2, h2)

def mse_loss(y_hat, y_true):
    return 0.5 * (y_hat - y_true) ** 2 # MSE loss

# Backward pass
def backward(y_hat, y_true, params, cache):
    x, a1, h1, a2, h2 = cache
    W1, W2, W3 = params["W1"], params["W2"], params["W3"]
    # output layer Gradient->Second layer Gradient->First layer
Gradient
    # output layer
    dL_dyhat = (y_hat - y_true)
    dW3 = dL_dyhat * h2.reshape(1, -1)
    db3 = dL_dyhat
    # second layer
    dL_dh2 = (W3.T * dL_dyhat).reshape(-1)
    dL_da2 = dL_dh2 * (a2 > 0)
    dW2 = np.outer(dL_da2, h1)
    db2 = dL_da2
    # first layer
    dL_dh1 = W2.T @ dL_da2
    dL_da1 = dL_dh1 * (a1 > 0)
    dW1 = np.outer(dL_da1, x)
    db1 = dL_da1

    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2,
            "W3": dW3, "b3": db3}
# Adam optimizer
def init_adam_state(params):
    return {f"m_{k}": np.zeros_like(v) for k, v in params.items()} | \
           {f"v_{k}": np.zeros_like(v) for k, v in params.items()}
```

```
def adam_update(params, grads, adam_state, t, lr=0.001,
                beta1=0.9, beta2=0.999, eps=1e-8):

    for k in params.keys():
        g = grads[k]
        m = adam_state[f"m_{k}"]
        v = adam_state[f"v_{k}"]

        m = beta1 * m + (1 - beta1) * g
        v = beta2 * v + (1 - beta2) * (g * g)

        m_hat = m / (1 - beta1 ** t)
        v_hat = v / (1 - beta2 ** t)

        params[k] -= lr * m_hat / (np.sqrt(v_hat) + eps)
        adam_state[f"m_{k}"] = m
        adam_state[f"v_{k}"] = v

    return params, adam_state
```